

Module 3: IOT and M2M with System Management

IOT and M2M: Introduction: M2M, Difference between IoT and M2M, SDN and NFV for IOT, IOT System Management with NETCONF-YANG, Need for IOT Systems Management, Simple Network Management Protocol (SNMP).

Textbooks:

1. Internet of Things: A Hands-On Approach, by Arshdeep Bahga & Vijay Madisetti, Universities Press, 2014

Reference books:

1. Introduction to IoT, by Sudip Misra, Anandarup Mukherjee, Arijit Roy, Cambridge University Press 2021
2. IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things – David Hanes, Gonzalo Salgueiro, Patrick Grossetete, Pearson Education, 2017

INTRODUCTION

Machine-to-machine (M2M) is a technology that uses a device attached to a machine to capture an event which is relayed through a mobile phone or fixed line network to an application that translates the event into meaningful information. The Internet of Things (IoT) is the next generation of the Internet based on the Internet Protocol (IP).

- Term which is often synonymous with IoT is Machine-to-Machine (M2M).
- IoT and M2M are often used interchangeably.

M2M Communication

- Machine-to-machine communication, or M2M, is exactly as it sounds: two machines “communicating,” or exchanging data, without human interfacing or interaction.
- This includes serial connection, powerline connection (PLC), or wireless communications in the industrial Internet of Things (IoT).
- Switching over to wireless has made M2M communication much easier and enabled more applications to be connected. In general, when someone says M2M communication, they often are referring to cellular communication for embedded devices.

- Examples of M2M communication in this case would be vending machines sending out inventory information or ATM machines getting authorization to dispense cash. As businesses have realized the value of M2M, it has taken on a new name: The Internet of Things (IoT).
- IoT and M2M have similar promises: to fundamentally change the way the world operates. Just like IoT, M2M allows virtually any sensor to communicate, which opens up the possibility of systems monitoring themselves and automatically responding to changes in the environment, with a much-reduced need for human involvement.
- M2M and IoT are almost synonymous—the exception is IoT (the newer term) typically refers to wireless communications, whereas M2M can refer to any two machines—wired or wireless communicating with one another.

Traditionally, M2M focused on “industrial telematics,” which is a fancy way of explaining data transfer for some commercial benefit. But many original uses of M2M still stand today, like smart meters. Wireless M2M has been dominated by cellular since it came out in the mid-2000’s with 2G cell networks. Because of this, the cellular market has tried to brand M2M as an inherently cellular thing by offering M2M data plans. But cellular M2M is only one subsection of the market, and it shouldn’t be thought of as a cellular-only area.

How M2M Works

As previously stated, machine-to-machine communication makes the Internet of Things possible. According to Forbes, M2M is among the fastest-growing types of connected device technologies in the market right now, largely because M2M technologies can connect millions of devices within a single network. The range of connected devices includes anything from vending machines to medical equipment to vehicles to buildings. Virtually anything that houses sensor or control technology can be connected to some sort of wireless network.

This sounds complex, but the driving thought behind the idea is quite simple. Essentially, M2M networks are very similar to LAN or WAN networks, but are exclusively used to allow machines, sensors, and controls, to communicate. These devices feed information they collect back to other devices in the network. This process allows a human (or an intelligent control unit) to assess what is going on across the whole network and issue appropriate instructions to member devices.

M2M Applications

The possibilities in the realm of M2M can be seen in four major use cases, which we've detailed below:

1. MANUFACTURING

Every manufacturing environment—whether it's food processing or general product manufacturing—relies on technology to ensure costs are managed properly and processes are executed efficiently. Automating manufacturing processes within such a fast-paced environment is expected to improve processes even more. In the manufacturing world, this could involve highly automated equipment maintenance and safety procedures.

For example, M2M tools allow business owners to be alerted on their smartphones when an important piece of equipment needs servicing, so they can address issues as quickly as they arise. Sophisticated networks of sensors connected to the Internet could even order replacement parts automatically.

2. HOME APPLIANCES

IoT already affects home appliance connectivity through platforms like Nest. However, M2M is expected to take home-based IoT to the next level. Manufacturers like LG and Samsung are already slowly unveiling smart home appliances to help ensure a higher quality of life for occupants.

For example, an M2M-capable washing machine could send alerts to the owners' smart devices once it finishes washing or drying, and a smart refrigerator could automatically order groceries from Amazon once its inventory is depleted. There are many more examples of home automation that can potentially improve quality of life for residents, including systems that allow members of the household to remotely control HVAC systems using their mobile devices. In situations where a homeowner decides to leave work early, he or she could contact the home heating system before leaving work to make sure the temperature at home will be comfortable upon arrival.

3. HEALTHCARE DEVICE MANAGEMENT

One of the biggest opportunities for M2M technology is in the realm of health care. With M2M technology, hospitals can automate processes to ensure the highest levels of treatment. Using devices that can react faster than a human healthcare professional in an emergency situation make this possible. For instance, when a patient's vital signs drop below normal, an M2M-connected life support device could automatically administer oxygen and additional care until a healthcare professional arrives on the scene. M2M also allows patients to be monitored in their own homes instead of in hospitals or care centers.

For example, devices that track a frail or elderly person's normal movement can detect when he or she has had a fall and alert a healthcare worker to the situation.

4. SMART UTILITY MANAGEMENT

In the new age of energy efficiency, automation will quickly become the new normal. As energy companies look for new ways to automate the metering process, M2M comes to the rescue, helping energy companies automatically gather energy consumption data, so they can accurately bill customers. Smart meters can track how much energy a household or business uses and automatically alert the energy company, which supplants sending out an employee to read the meter or requiring the customer to provide a reading. This is even more important as utilities move toward more dynamic pricing models, charging consumers more for energy usage during peak times. A few key analysts predict that soon, every object or device will need to be able to connect to the cloud. This is a bold but seemingly accurate statement. As more consumers, users, and business owners demand deeper connectivity, technology will need to be continually equipped to meet the needs and challenges of tomorrow. This will empower a wide range of highly automated processes, from equipment repairs and firmware upgrades to system diagnostics, data retrieval, and analysis. Information will be delivered to users, engineers, data scientists, and key decision-makers in real time, and it will eliminate the need for guesswork.

There are different M2M applications, environment monitoring, civil protection and public safety, supply chain management, energy and utility distribution as in smart grid, smart grid separately common. we have intelligent transportation systems, healthcare, automation of buildings, military applications, agriculture, home networks all these are different applications of M2M.

M2M features:

- Large number of nodes or devices
- Low cost
- Energy efficient
- Small traffic per device/machine
- M2M communication free from human intervention

General Architecture of M2M Systems:

- M2M device connects to the network domain via direct connectivity or M2M gateway. In the first case, the M2M device connects to the network domain via the access network, which performs the procedures such as registration, authentication, authorization, management, and provisioning with the network domain. In the second case, the M2M device connects to the M2M gateway using the M2M area network.
- M2M area network provides connectivity between M2M devices and M2M gateways.
- M2M gateway acts as a proxy between M2M devices and the network domain. As an example, an M2M gateway can run an application that collects and treats various information (e.g., contextual parameters) from sensors and meters.
- M2M communication network provides connection between the M2M gateways/devices and the M2M servers. Usually it contains two parts: the access network and the Internet.
- M2M server works as a middleware layer to pass data through various application services.

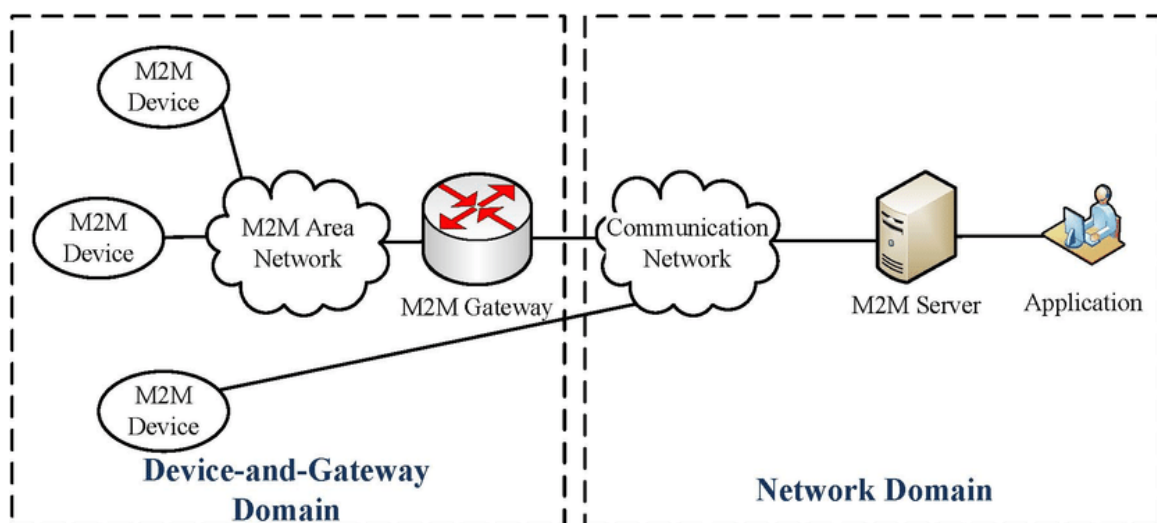


Figure: General Architecture of M2M Systems

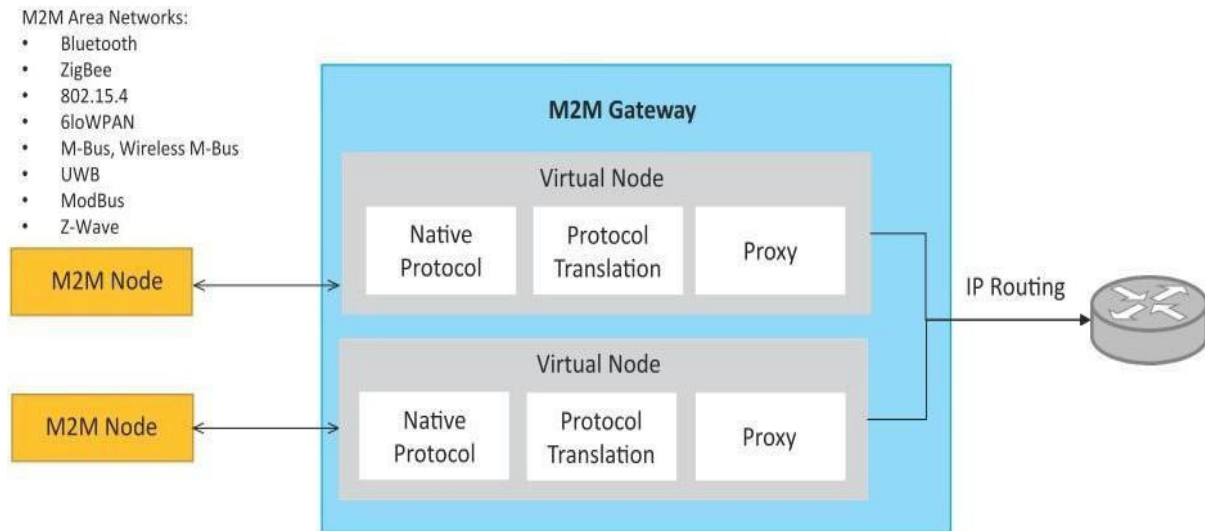


Figure: M2M Gate way Architecture

Difference Between IoT and M2M:

M2M, or machine-to-machine, is a direct communication between devices using wired or wireless communication channels. M2M refers to the interaction of two or more devices/machines that are connected to each other. These devices capture data and share with other connected devices, creating an intelligent network of things or systems. Devices could be sensors, actuators, embedded systems or other connected elements.

M2M technology could be present in our homes, offices, shopping malls and other places. Controlling electrical appliances like bulbs and fans using RF or Bluetooth from your smartphone is a simple example of M2M applications at home. Here, the electrical appliance and your smartphone are the two machines interacting with each other.

The Internet of Things (IoT) is the network of physical devices embedded with sensors, software and electronics, enabling these devices to communicate with each other and exchange data over a computer network. The things in the IoT refer to hardware devices uniquely identifiable through a network platform within the Internet infrastructure.

M2M versus the IoT

M2M	IoT
M2M is about direct communication between machines.	The IoT is about sensors automation and Internet platform.
It supports point-to-point communication.	It supports cloud communication.
Devices do not necessarily rely on an Internet connection.	Devices rely on an Internet connection.
M2M is mostly hardware-based technology.	The IoT is both hardware- and software-based technology.
Machines normally communicate with a single machine at a time.	Many users can access at one time over the Internet.
A device can be connected through mobile or other network.	Data delivery depends on the Internet protocol (IP) network.

Some more differences like:

Communication Protocols:

- M2M and IoT can differ in how the communication between the machines or devices happens.
- M2M uses either proprietary or non-IP based communication protocols for

communication within the M2M area networks. IoT uses IP based communication protocols.

Machines in M2M vs Things in IoT:

- The "Things" in IoT refers to physical objects that have unique identifiers and can sense and communicate with their external environment (and user applications) or their internal physical states.
- M2M systems, in contrast to IoT, typically have homogeneous machine types within an M2M area network.

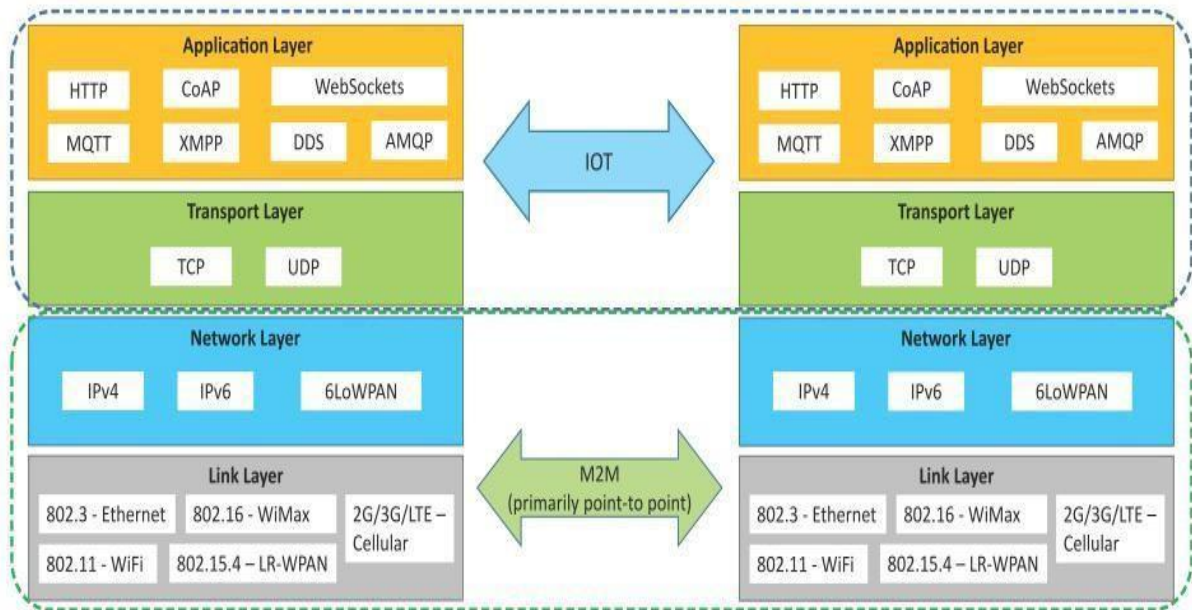


Figure: Communication in IoT vs M2M

Hardware vs Software Emphasis:

- While the emphasis of M2M is more on hardware with embedded modules, the emphasis of IoT is more on software.

Data Collection & Analysis:

- M2M data is collected in point solutions and often in on-premises storage infrastructure.
- In contrast to M2M, the data in IoT is collected in the cloud (can be public, private or hybrid cloud).

Applications:

- M2M data is collected in point solutions and can be accessed by on premises applications such as diagnosis applications, service management applications, and on-premises enterprise applications.
- IoT data is collected in the cloud and can be accessed by cloud applications such as analytics applications, enterprise applications, remote diagnosis and management applications, etc

IoT Levels Summary

IoT Level	Deployment Description	Main Processing Location	Typical Cloud Components Used	Example Application
Level 1 – Device Level	Single node performs sensing, processing, and actuation. No cloud connection.	Local device	None (no cloud)	Smart door lock, local temperature monitor
Level 2 – Device-to-Cloud	One IoT device connected to the cloud for storage or remote monitoring.	Device + Cloud	Cloud storage, cloud app services, APIs / REST interface	Fitness tracker uploading data
Level 3 – Device-to-Gateway-to-Cloud	IoT devices send data to a local gateway for aggregation before cloud upload.	Gateway (Edge)	Data services, Device management, MQTT/AMQP middleware, Security/authentication	Smart home network with multiple sensors
Level 4 – Device–Edge–Cloud Hybrid	Devices + edge servers + centralized cloud. Application logic divided.	Edge + Cloud	Cloud storage, Security, Analytics, Monitoring	Smart agriculture (edge + cloud dashboard)
Level 5 – Distributed Cloud	Multiple clouds/federated systems. Large scale, distributed architecture.	Multi-cloud / Distributed	Cloud federation, Interoperability APIs, Load balancing, Fault tolerance	Smart city platform combining multiple services
Level 6 –	Multiple	Fully	Big data analytics,	Global

Complex Distributed IoT System	independent subsystems, autonomous operation, global coordination.	distributed	AI/ML, Multi-cloud interoperability, Lifecycle mgmt	logistics tracking, industrial IoT network
--------------------------------	--	-------------	---	--

IoT Cloud / Back-End Components

Cloud Component	Function
Cloud Storage Services	Store large-scale sensor and event data.
Application Services / Logic	Business logic, automation rules, dashboards.
Data Analytics & Visualization	Batch/stream processing, dashboards, ML.
APIs & Communication Interface	REST, WebSocket, MQTT, CoAP for device-cloud interaction.
Security & Identity Management	Device authentication, authorization, encryption.
Device Management	Provisioning, firmware updates, monitoring.
Monitoring & Diagnostics	Logging, alerts, performance monitoring.
Scalability & Fault Tolerance	Load balancing, redundancy, high availability.
Integration / Interoperability	Linking multiple systems or cloud services.
Federation & multi-cloud	Cross-cloud coordination, shared data, distributed control.

Software defined Networking (SDN)

SDN stands for Software Defined Network which is a networking architecture approach. It enables the control and management of the network using software applications. Through

Software Defined Network (SDN) networking behavior of the entire network and its devices are programmed in a centrally controlled manner through software applications using open APIs.

To understand software-defined networks, we need to understand the various planes involved in networking.

1. Data Plane
2. Control Plane

Data plane:

All the activities involving as well as resulting from data packets sent by the end-user belong to this plane. This includes:

- Forwarding of packets.
- Segmentation and reassembly of data.
- Replication of packets for multicasting.

Control plane:

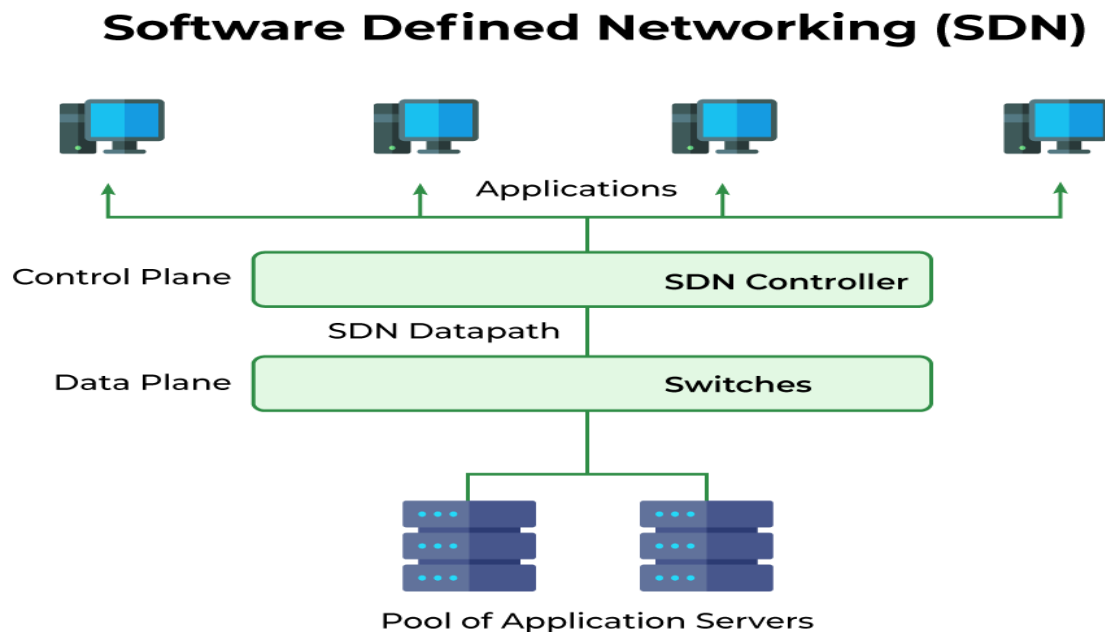
All activities necessary to perform data plane activities but do not involve end-user data packets belong to this plane. In other words, this is the brain of the network. The activities of the control plane include:

- Making routing tables.
- Setting packet handling policies.

Why SDN is Important?

- **Better Network Connectivity:** SDN provides very better network connectivity for sales, services, and internal communications. SDN also helps in faster data sharing.
- **Better Deployment of Applications:** Deployment of new applications, services, and many business models can be speed up using Software Defined Networking.
- **Better Security:** Software-defined network provides better visibility throughout the network. Operators can create separate zones for devices that require different levels of security. SDN networks give more freedom to operators.
- **Better Control with High Speed:** Software-defined networking provides better speed than other networking types by applying an open standard software-based controller.

In short, it can be said that- SDN acts as a “Bigger Umbrella or a HUB” where the rest of other networking technologies come and sit under that umbrella and get merged with another platform to bring out the best of the best outcome by decreasing the traffic rate and by increasing the efficiency of data flow.



Where is SDN Used?

- Enterprises use SDN, the most widely used method for application deployment, to deploy applications faster while lowering overall deployment and operating costs. SDN allows IT administrators to manage and provision network services from a single location.
- Cloud networking software-defined uses white-box systems. Cloud providers often use generic hardware so that the Cloud data center can be changed and the cost of CAPEX and OPEX saved.

Components of Software Defining Networking (SDN)

The three main components that make the SDN are:

1. **SDN Applications:** SDN Applications relay requests or networks through SDN Controller using API.

2. **SDN controller:** SDN Controller collects network information from hardware and sends this information to applications.
3. **SDN networking devices:** SDN Network devices help in forwarding and data processing tasks.

SDN Architecture

In a traditional network, each switch has its own data plane as well as the control plane. The control plane of various switches exchange topology information and hence construct a forwarding table that decides where an incoming data packet has to be forwarded via the data plane. Software-defined networking (SDN) is an approach via which we take the control plane away from the switch and assign it to a centralized unit called the SDN controller. Hence, a network administrator can shape traffic via a centralized console without having to touch the individual switches. The data plane still resides in the switch and when a packet enters a switch, its forwarding activity is decided based on the entries of flow tables, which are pre-assigned by the controller. A flow table consists of match fields (like input port number and packet header) and instructions. The packet is first matched against the match fields of the flow table entries. Then the instructions of the corresponding flow entry are executed. The instructions can be forwarding the packet via one or multiple ports, dropping the packet, or adding headers to the packet. If a packet doesn't find a corresponding match in the flow table, the switch queries the controller which sends a new flow entry to the switch. The switch forwards or drops the packet based on this flow entry.

A typical SDN architecture consists of three layers.

- **Application layer:** It contains the typical network applications like intrusion detection, firewall, and load balancing.
- **Control layer:** It consists of the SDN controller which acts as the brain of the network. It also allows hardware abstraction to the applications written on top of it.
- **Infrastructure layer:** This consists of physical switches which form the data plane and carries out the actual movement of data packets.

The layers communicate via a set of interfaces called the north-bound APIs (between the application and control layer) and southbound APIs (between the control and infrastructure layer).

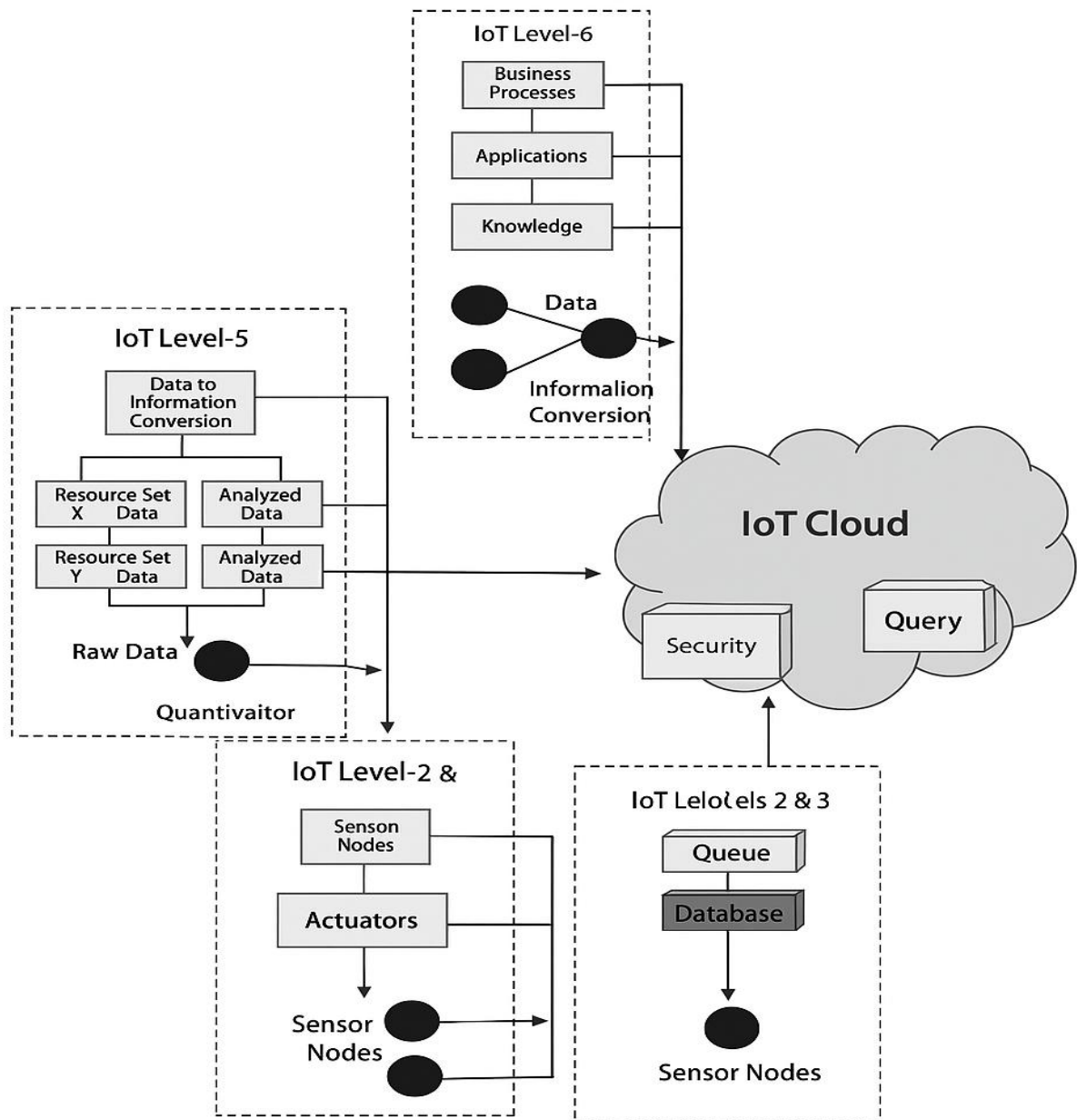


Figure 3.5: IoT levels and IoT cloud components

In this section you will learn about Software Defined Networking (SDN) and Network Function Virtualization *for* IoT.

Different Models of SDN

There are several models, which are used in SDN:

1. Open SDN
 2. SDN via APIs
 3. SDN via Hypervisor-based Overlay Network
 4. Hybrid SDN
- 1. Open SDN:** Open SDN is implemented using the OpenFlow switch. It is a straightforward implementation of SDN. In Open SDN, the controller communicates with the switches using south-bound API with the help of OpenFlow protocol.

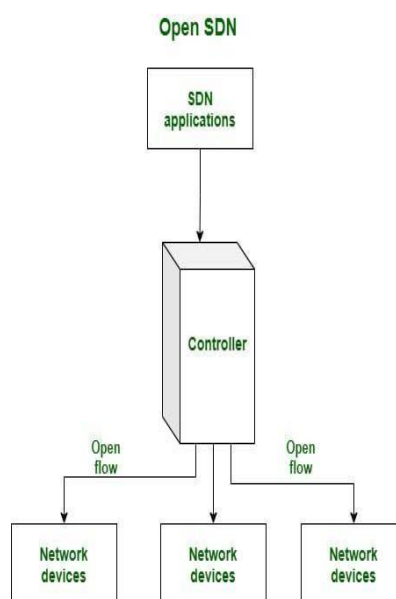
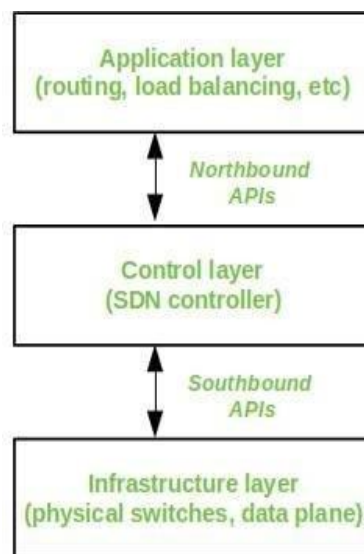


Fig: Open SDN

- c. SDN architecture supports programmable open APIs for interface between the SDN application and control layers (Northbound interface).
- d. Standard Communication Interface (OpenFlow)
- e. SDN architecture uses a standard communication interface between the control and infrastructure layers (Southbound interface).

Difference between SDN and Traditional Networking

Software Defined Networking	Traditional Networking
Software Defined Network is a virtual networking approach.	A traditional network is the old conventional networking approach.
Software Defined Network is centralized control.	Traditional Network is distributed control.
This network is programmable.	This network is nonprogrammable.
Software Defined Network is the open interface.	A traditional network is a closed interface.
In Software Defined Network data plane and control, the plane is decoupled by software.	In a traditional network data plane and control plane are mounted on the same plane.

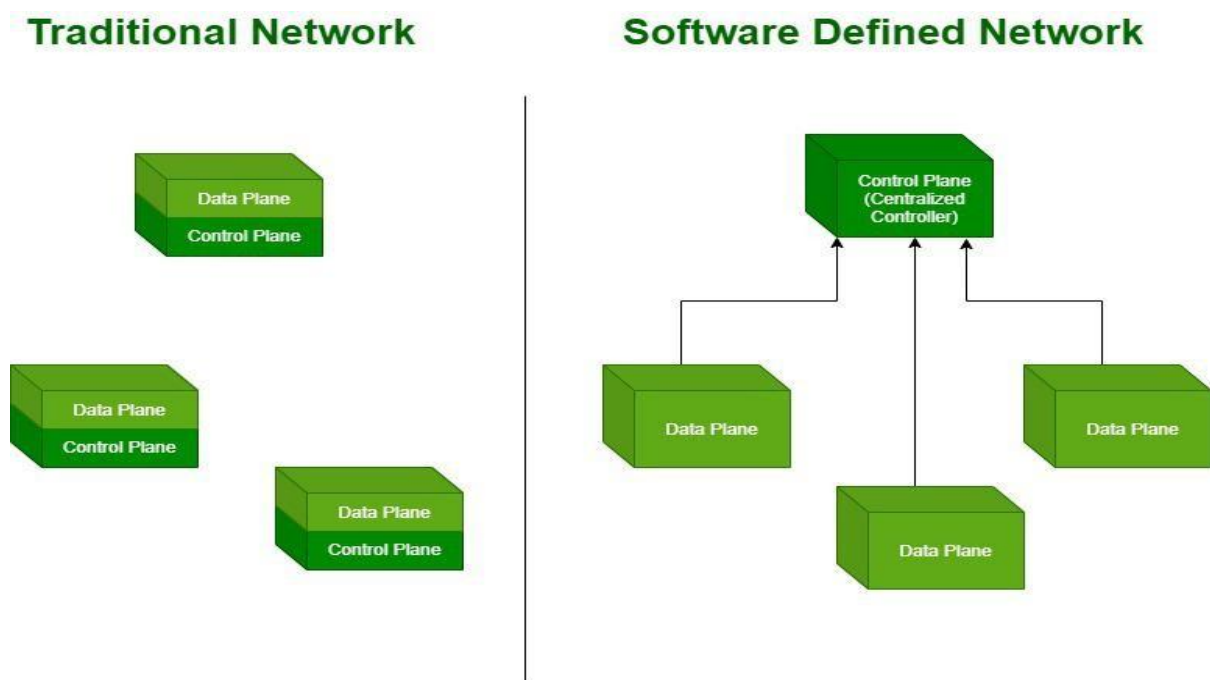


Figure : Difference between SDN and Traditional Networking

Advantages of SDN

The network is programmable and hence can easily be modified via the controller rather than individual switches.

- Switch hardware becomes cheaper since each switch only needs a data plane.
- Hardware is abstracted hence applications can be written on top of the controller independent of the switch vendor.
- Provides better security since the controller can monitor traffic and deploy security policies. For example, if the controller detects suspicious activity in network traffic, it can reroute or drop the packets.

Disadvantages of SDN

- The central dependency of the network means a single point of failure, i.e. if the controller gets corrupted, the entire network will be affected.
- The use of SDN on large scale is not properly defined and explored.

Network Functions Virtualization

The term “Network Functions Virtualization” (NFV) refers to the use of virtual machines in place of physical network appliances. There is a requirement for a hypervisor to operate networking software and procedures like load balancing and routing by virtual computers. A network functions virtualization standard was first proposed at the OpenFlow World Congress in 2012 by the European Telecommunications Standards Institute (ETSI), a group of service providers that includes AT&T, China Mobile, BT Group, Deutsche Telekom, and many more.

Need of NFV:

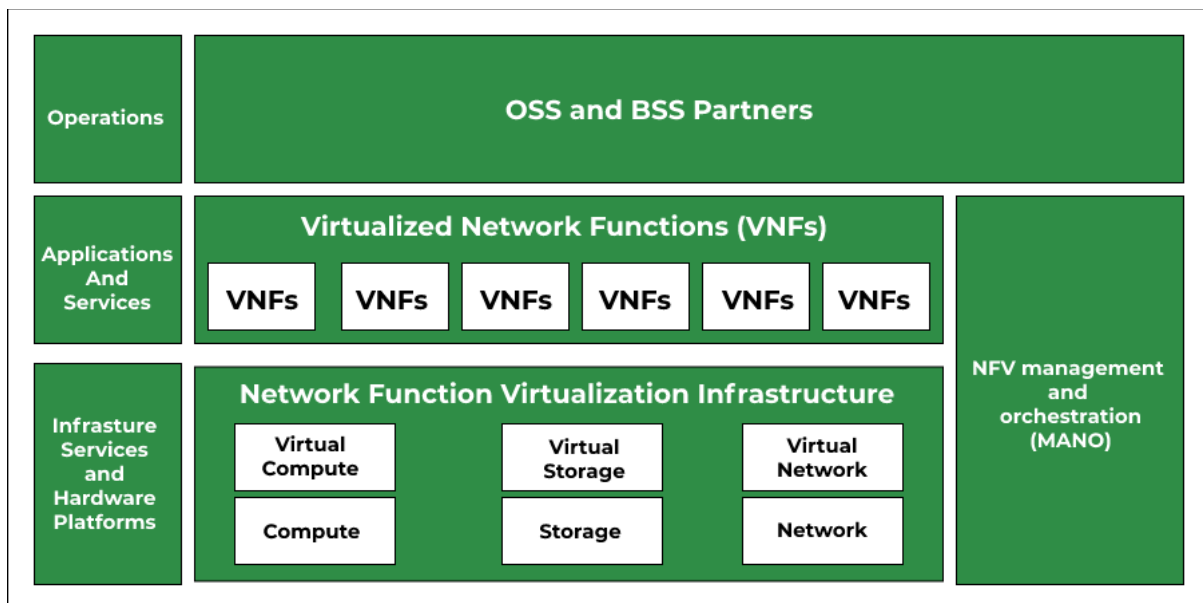
With the help of NFV, it becomes possible to separate communication services from specialized hardware like routers and firewalls. This eliminates the need for buying new hardware and network operations can offer new services on demand. With this, it is possible to deploy network components in a matter of hours as opposed to months as with conventional networking. Furthermore, the virtualized services can run on less expensive generic servers.

Advantages:

- Lower expenses as it follows Pay as you go which implies companies only pay for what they require.
- Less equipment as it works on virtual machines rather than actual machines which leads to fewer appliances, which lowers operating expenses as well.
- Scalability of network architecture is quite quick and simple using virtual functions in NFV. As a result, it does not call for the purchase of more hardware.

Working:

Usage of software by virtual machines enables to carry out the same networking tasks as conventional hardware. The software handles the task of load balancing, routing, and firewall security. Network engineers can automate the provisioning of the virtual network and program all of its various components using a hypervisor or software-defined networking controller.



Benefits of NFV:

- Many service providers believe that advantages outweigh the issues of NFV.
- Traditional hardware-based networks are time-consuming as these require network administrators to buy specialized hardware units, manually configure them, then join them to form a network. For this skilled or well-equipped worker is required.

- It costs less as it works under the management of a hypervisor, which is significantly less expensive than buying specialized hardware that serves the same purpose.
- Easy to configure and administer the network because of a virtualized network. As a result, network capabilities may be updated or added instantly.

Risks of NFV:

Security hazards do exist, though, and network functions virtualization security issues have shown to be a barrier to widespread adoption among telecom companies. The following are some dangers associated with implementing network function virtualization that service providers should take into account:

- Physical security measures do not work: Comparing virtualized network components to locked-down physical equipment in a data center enhances their susceptibility to new types of assaults.
- Malware is difficult to isolate and contain: Malware travels more easily among virtual components running on the same virtual computer than between hardware components that can be isolated or physically separated.
- Network activity is less visible: Because traditional traffic monitoring tools struggle to detect potentially malicious anomalies in network traffic going east-west between virtual machines, NFV necessitates more fine-grained security solutions.

NFV Architecture:

An individual proprietary hardware component, such as a router, switch, gateway, firewall, load balancer, or intrusion detection system, performs a specific networking function in a typical network architecture. A virtualized network substitutes software programs that operate on virtual machines for these pieces of hardware to carryout networking operations.

Three components make up an NFV architecture:

- Centralized virtual network infrastructure: The foundation of an NFV infrastructure can be either a platform for managing containers or a hypervisor that abstracts the resources for computation, storage, and networking.
- Applications: Software delivers many forms of network functionality by substituting for the hardware elements of a conventional network design (virtualized network functions).

- Framework: To manage the infrastructure and provide network functionality, a framework is required (commonly abbreviated as MANO, meaning Management, Automation, and Network Orchestration).

Examples hows how NFV can be used for the virtualization of home networks

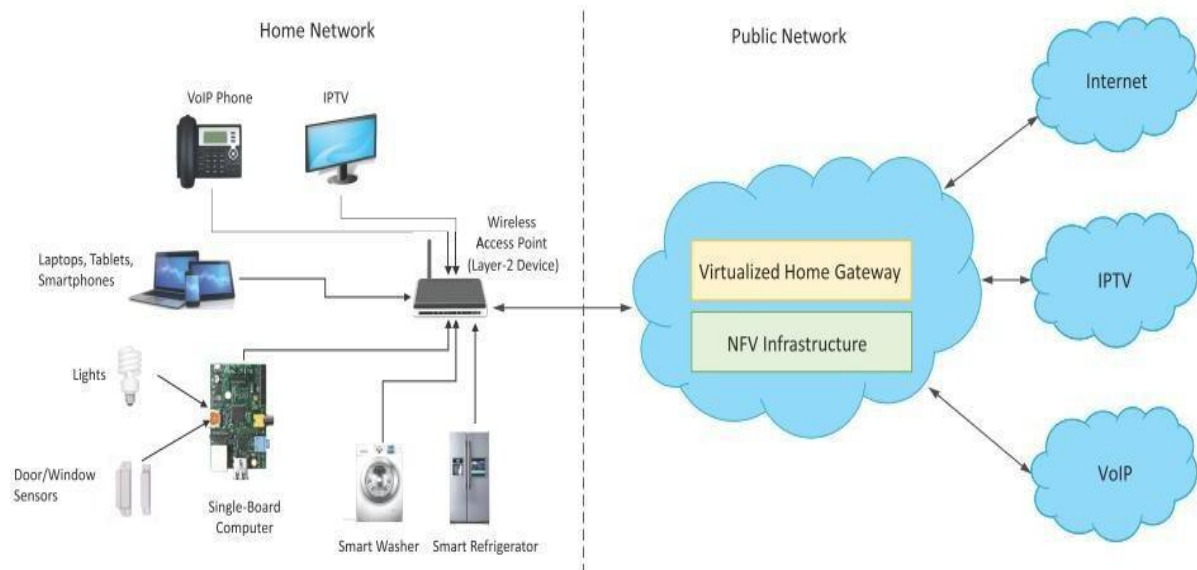


Figure: NFV virtualization of home networks

Need for IoT Systems Management

Managing multiple devices within a single system requires advanced management capabilities.

- 1. Automating Configuration:** IoT system management capabilities can help in automating the system configuration.
- 2. Monitoring Operational & Statistical Data:** Management systems can help in monitoring operational and statistical data of a system. This data can be used for fault diagnosis or prognosis.
- 3. Improved Reliability:** A management system that allows validating the system configurations before they are put into effect can help in improving the system reliability.

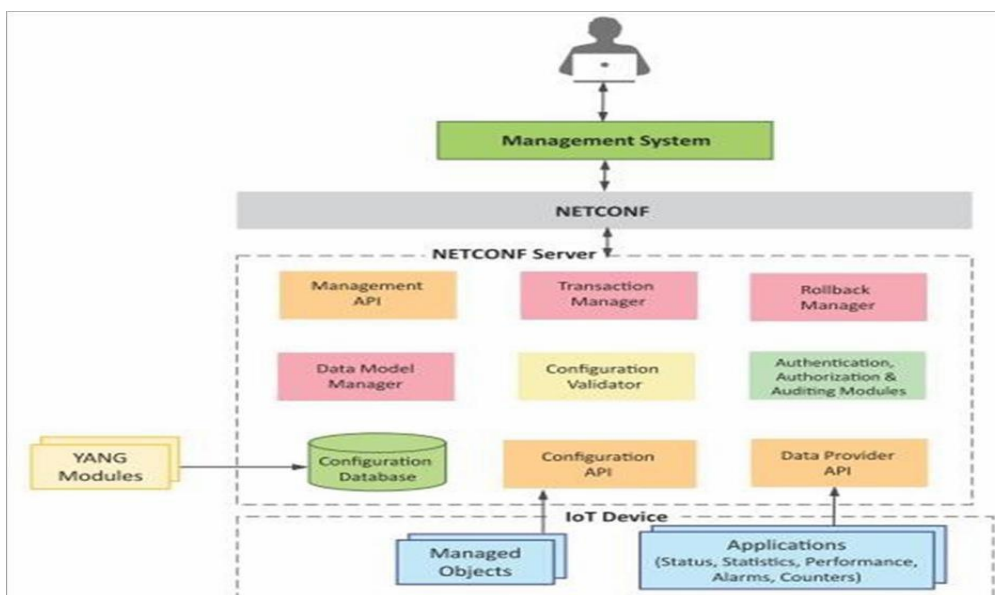
4. **System Wide Configurations:** For IoT systems that consists of multiple devices or nodes, ensuring system wide configuration can be critical for the correct functioning of the system.
5. **Multiple System Configurations:** For some systems it may be desirable to have multiple valid configurations which are applied at different times or in certain conditions.
6. **Retrieving & Reusing Configurations:** Management systems which have the capability of retrieving configurations from devices can help in reusing the configurations for other devices of the same type.

IoT Systems Management with NETCONF-YANG

YANG is a data modelling language used to model configuration and stated at a manipulated by the NETCONF protocol.

The generic approach of IoT device management with NETCONF-YANG. Roles of various components are:

1. Management System
2. Management API
3. Transaction Manager
4. Rollback Manager
5. Data Model Manager
6. Configuration Validator
7. Configuration Database
8. Configuration API
9. Data Provider API



1. **Management System:** The operator uses a management system to send NETCONF messages to configure the IoT device and receives state information and notifications from the device as NETCONF messages.
2. **Management API:** allows management application to start NETCONF sessions.
3. **Transaction Manager:** executes all the NETCONF transactions and ensures that ACID properties hold true for the transactions.
4. **Rollback Manager:** is responsible for generating all the transactions necessary to rollback a current configuration to its original state.
5. **Data Model Manager:** Keeps track of all the YANG data models and the corresponding managed objects. Also keeps track of the applications which provide data for each part of a data model.
6. **Configuration Validator:** checks if the resulting configuration after applying a transaction would be a valid configuration.
7. **Configuration Database:** contains both configuration and operational data.
8. **Configuration API:** Using the configuration API the application on the IoT device can be read configuration data from the configuration datastore and write operational data to the operational datastore.
9. **Data Provider API:** Applications on the IoT device can register for callbacks for various events using the Data Provider API. Through the Data Provider API, the applications can report statistics and operational data.

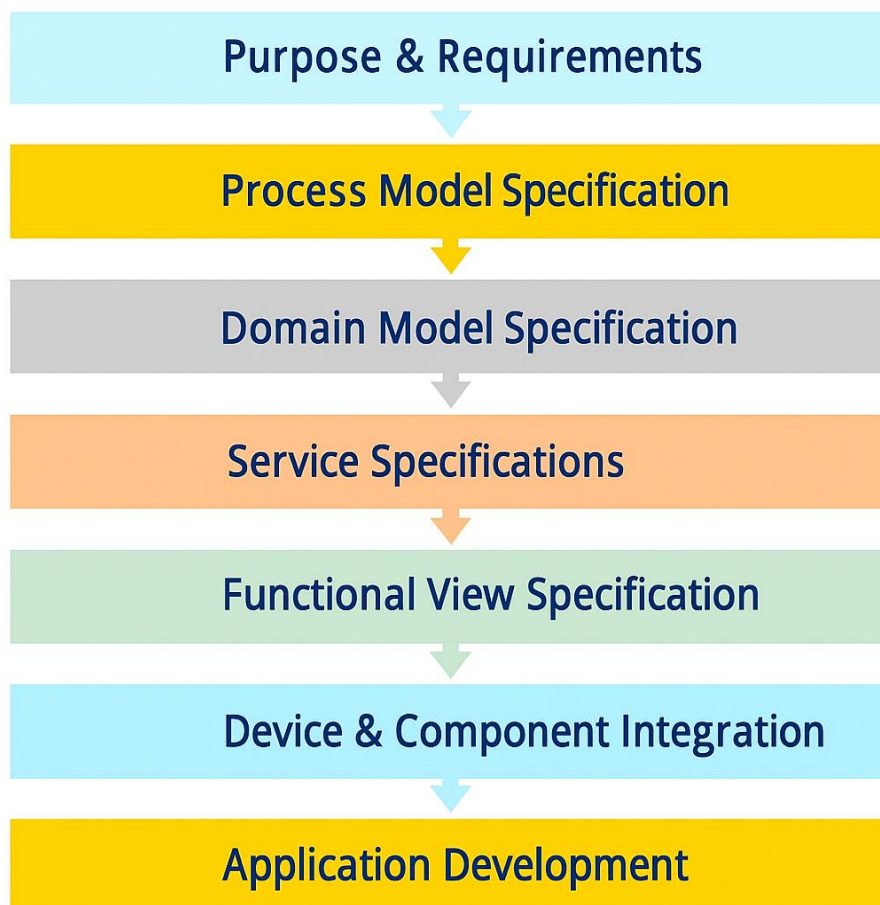
YANG Module Example

- This YANG module is a YANG version of the toaster MIB
- The toaster YANG module begins with the header information followed by identity declarations which define various bread types.
- The leaf nodes ('toasterManufacturer',
- 'toasterModelNumber' and oasterStatus') are defined in the 'toaster' container.
- Each leaf node definition has a type and optionally a description and default value.
- The module has two RPC definitions ('make-toast' and 'cancel-toast').

Steps for IoT device Management with NETCONF-YANG

1. Create a YANG model of the system that defines the configuration and state data of the system.
2. Complete the YANG model with the Inctool which comes with Libnetconf.
3. Fill in the IoT device management code in the TransAPI module.
4. Build the callbacks C file to generate the library file.
5. Load the YANG module and the TransAPI module into the Netopeer server using Netopeer manager tool.
6. The operator can now connect from the management system to the Netopeer server using the Netopeer CLI.
7. Operator can issue NETCONF commands from the Netopeer CLI. Command can be issued to change the configuration data, get operational data or execute an RPC on the IoT device.

IoT Design Methodology – IoT device Management Steps



Step 1: Purpose & Requirements Specification:

The first step in IoT system design methodology is to define the purpose and requirements of the system. In this step, the system purpose, behavior and requirements (such as data collection requirements, data analysis requirements, system management requirements, data privacy and security requirements, user interface requirements, ...) are captured.

Step 2: Process Specification:

The second step in the IoT design methodology is to define the process specification. In this step, the use cases of the IoT system are formally described based on and derived from the purpose and requirement specifications.

Step 3: Domain Model Specification:

The third step in the IoT design methodology is to define the Domain Model. The domain model describes the main concepts, entities and objects in the domain of IoT system to be designed. Domain model defines the attributes of the objects and relationships between objects. Domain model provides an abstract representation of the concepts, objects and entities in the IoT domain, independent of any specific technology or platform. With the domain model, the IoT system designers can get an understanding of the IoT domain for which the system is to be designed.

Step 4: Information Model Specification:

The fourth step in the IoT design methodology is to define the Information Model. Information Model defines the structure of all the information in the IoT system, for example, attributes of Virtual Entities, relations, etc. Information model does not describe the specifics of how the information is represented or stored. To define the information model, we first list the Virtual Entities defined in the Domain Model. Information model adds more details to the Virtual Entities by defining their attributes and relations.

Step 5: Service Specifications:

The fifth step in the IoT design methodology is to define the service specifications. Service specifications define the services in the IoT system, service types, service

inputs/output, service endpoints, service schedules, service preconditions and service effects.

Step 6: IoT Level Specification:

The sixth step in the IoT design methodology is to define the IoT level for the system.

Step 7: Functional View Specification:

The seventh step in the IoT design methodology is to define the Functional View. The Functional View (FV) defines the functions of the IoT systems grouped into various Functional Groups (FGs). Each Functional Group either provides functionalities for interacting with instances of concepts defined in the Domain Model or provides information related to these concepts.

Step 8: Operational View Specification:

The eighth step in the IoT design methodology is to define the Operational View Specifications. In this step, various options pertaining to the IoT system deployment and operation are defined, such as, service hosting options, storage options, device options, application hosting options, etc

Step 9: Device & Component Integration:

The ninth step in the IoT design methodology is the integration of the devices and components.

Step 10: Application Development:

The final step in the IoT design methodology is to develop the IoT application.

SNMP

The Simple Network Management Protocol (SNMP) is an application-layer protocol used to manage and monitor network devices such as routers, switches, servers, printers, and firewalls. It is a foundational component of modern network management systems, allowing administrators to gather performance data, detect network faults, and configure devices remotely.

How SNMP works

The SNMP framework uses a manager-agent model and is typically implemented over the User Datagram Protocol (UDP) on ports 161 (for manager-to-agent communication) and 162

(for agent-to-manager notifications).

The main components that work together are:

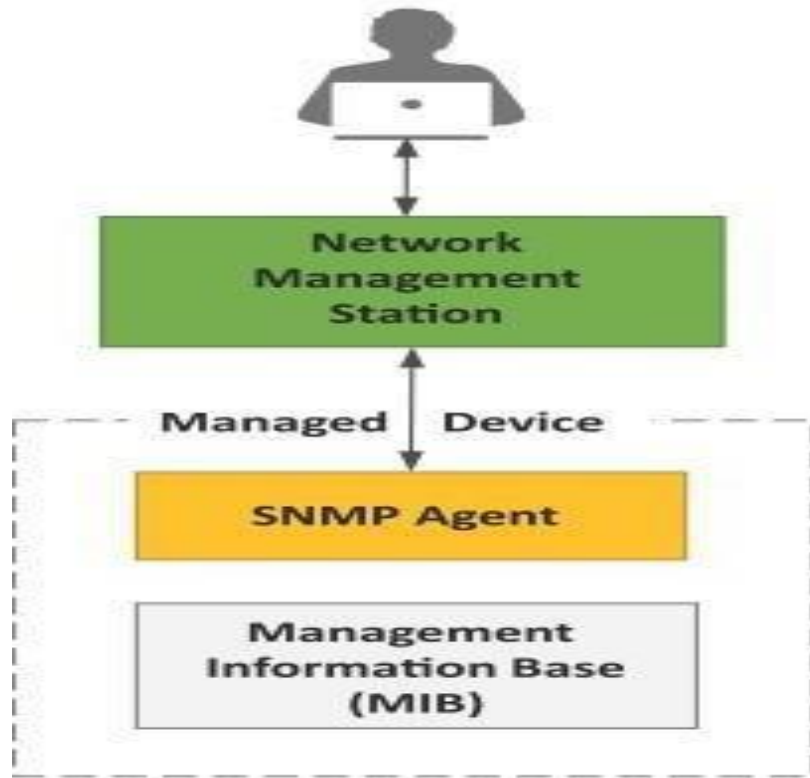


Figure: Managing Device with SNMP

SNMP Manager (Network Management System or NMS): A central system that runs monitoring applications. It communicates with SNMP agents to query devices for data, receive notifications, and send commands.

SNMP Agent: A software module that runs on a managed device. The agent collects information about the device's status and performance and responds to requests from the SNMP manager.

- **Managed Devices:** Any network device that is running an SNMP agent and is being monitored. This can include routers, switches, printers, and servers.
- **Management Information Base (MIB):** A database that formally describes the managed objects on a device.
- It organizes data hierarchically and allows the manager to understand the information sent by the agent.

- **Object Identifier (OID):** A unique alphanumeric string that identifies a specific managed object within the
- MIB hierarchy. Managers use OIDs in their requests to specify exactly which piece of data they want to retrieve.

Key commands

Communication between the manager and agents is accomplished through several basic message types, known as Protocol Data Units (PDUs):

- **GetRequest:** The manager sends this command to an agent to retrieve the value of one or more specified object identifiers (OIDs).
- **SetRequest:** The manager sends this command to modify or assign a new value to a configurable parameter on a managed device.
- **GetResponse:** An agent's response to a GetRequest or SetRequest, containing the requested data or a confirmation that a change was successful.
- **Trap:** An asynchronous, unrequested notification sent by an agent to the manager when a significant event occurs, such as a device reboot or an error.
- **InformRequest:** Introduced in SNMPv2, this is similar to a trap, but it requires a response from the manager, confirming receipt.
- **GetBulkRequest:** Introduced in SNMPv2, this command efficiently retrieves large blocks of data, especially from MIB tables, using a single request.

SNMP versions

The protocol has evolved over time, primarily to address security concerns:

- **SNMPv1:** The original version, published in 1988, which uses a community string (a plaintext password) for authentication. It offers minimal security and is considered insecure for modern networks.
- **SNMPv2c:** An improved version that is widely used and compatible with the SNMPv1 security model. It adds enhancements like GetBulkRequest for better performance but still relies on community strings for authentication, making it vulnerable to interception.
- **SNMPv3:** The most secure version, which adds significant enhancements for authentication and encryption.

It prevents unauthorized access, tampering, and snooping of SNMP messages.

Uses and benefits

Network administrators rely on SNMP for a range of functions, including:

- **Performance monitoring:** Tracking metrics such as bandwidth usage, CPU load, and memory utilization to ensure devices are operating efficiently.
- **Fault management:** Receiving asynchronous alerts (traps) from devices when critical events occur, allowing for proactive troubleshooting.
- **Asset inventory:** Discovering and documenting devices on the network, including their hardware and firmware details.
- **Remote configuration:** Modifying device settings and configuration parameters from a central location.

Limitations of SNMP

- Simple Network Management Protocol (SNMP) has been the most popular protocol for network management, it has several limitations which may make it unsuitable for configuration management.
- SNMP is stateless in nature and each SNMP request contains all the information to process the request.
- The application needs to be intelligent to manage the device. For a sequence of SNMP interactions, the application needs to maintain state and also to be smart enough to roll back the device into a consistent state in case of errors or failures in configuration.
- SNMP is a connectionless protocol which uses UDP as the transport protocol making it unreliable as there was no support for acknowledgement of requests.
- MIBs often lack writable objects without which device configuration is not possible using SNMP.
- It is difficult to differentiate between configuration and state data in MIBs.
- Retrieving the current configuration from a device can be difficult with SNMP. SNMP does not support easy retrieval and playback of configurations.
- Earlier versions of SNMP did not have strong security features making the management information vulnerable to network intruders. Though security features were added in the later versions of SNMP it increased the complexity a lot.

Limitations of SNMP

- SNMP is stateless in nature and each SNMP request contains all the information to process the request. The application needs to be intelligent to manage the device.
- SNMP is a connectionless protocol which uses UDP as the transport protocol, making it unreliable as there was no support for acknowledgement of requests.
- MIBs often lack writable objects without which device configuration is not possible using SNMP.
- It is difficult to differentiate between configuration and state data in MIBs.
- Retrieving the current configuration from a device can be difficult with SNMP.
- Earlier versions of SNMP did not have strong security features.